Neural Networks 206 (2027) 109800

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

SemDiff: Binary similarity detection by diffing key-semantics graphs

Zian Liu[a,b,*], Xiaoxi Meng[a], Viet Vo[a], Siqi Ma[c], Chao Chen[d], Dongxi Liu[b], Jun Zhang[a], Shigang Liu[b], Muhammad Ejaz Ahmed[b], Yang Xiang[a]

[a] Swinburne University of Technology, 1 John Street, Melbourne, VIC, 3122, Australia
[b] Data 61, CSIRO, Corner Vimiera &, Pembroke Rd, Sydney, NSW, 2122, Australia
[c] University of New South Wales, Northcott Dr, Canberra, ACT, 2612, Australia
[d] Royal Melbourne Institution of Technology, 124 La Trobe Street, Melbourne, VIC, 3000, Australia

ARTICLE INFO

Keywords:
Binary code similarity detection
Vulnerability detection
Malware analysis

ABSTRACT

Binary similarity detection is a critical technique that has been applied in many real-world scenarios where source code is not available, e.g., bug search, malware analysis, and code plagiarism detection for executable binary code. Existing works are *ineffective* in detecting similar binaries in cases where different compiling optimizations, compilers, source code versions, or obfuscation are deployed.

We observe that all the cases do not change a binary's key code behaviors although they significantly modify its syntax and structure. With this key observation, we extract a set of *key* instructions from a binary to capture its key code behaviors. By detecting the similarity between two binaries' key instructions, we can address well the ineffectiveness limitation of existing works. Specifically, we translate each extracted key instruction into a self-defined key expression, generating a key-semantics graph based on the binary's control flow. Each node in the key-semantics graph denotes a key instruction, and the node attribute is the key expression. To quantify the similarity between two given key-semantics graphs, we first serialize each graph into a sequence of key expressions by topological sort. Then, we tokenize and concatenate key expressions to generate token lists. We calculate the locality-sensitive hash value for all token lists and quantify their similarity. Our evaluation results show that overall, SemDiff outperforms state-of-the-art tools when detecting the similarity of binaries generated from different optimization levels, compilers, and obfuscations. SemDiff is also effective for library version search and finding similar vulnerabilities in firmware.

1. Introduction

Binary code similarity detection (also known as binary diffing) is important for bug search, patch generation and analysis, malware detection, and plagiarism detection (Haq & Caballero, 2021). Existing works can be categorized into machine-learning-based approaches or program-analysis-based approaches.

Machine learning based approaches either capture syntactic, structural, and semantic features from binary code to train a similarity detection model (Cochard et al., 2022; Feng et al., 2016; Gao et al., 2018; Kim et al., 2022; Liu et al., 2018; Xu et al., 2017a; Yang et al., 2023; Yu et al., 2020) or leverage natural language processing (NLP) (Ding et al., 2016, 2019; Duan et al., 2020; Gu et al., 2025; Kim et al., 2023; Li et al., 2021; Liu et al., 2018; Massarelli et al., 2019; Wang et al., 2022; Zuo et al., 2022) to learn semantic information. Although such machine-learning-based approaches are theoretically effective, the performance highly relies on how well the training data is created. Real-world binaries are diverse, that is, the same piece of source code could be compiled into different pieces of binaries because of using different compilers (optimization levels) (Duan et al., 2020). It is thus difficult to build a representative dataset for training. If the training data are not well established, the corresponding approaches will be affected significantly. Unsupervised learning automatically learns each instruction embeddings from its context instructions. However, optimizations or different compilers can make changes to a non-trivial part of the function, rendering this method less accurate. On the other hand, machine learning-based methods aim to preserve semantic information by predicting assembly syntactic embeddings, though these embeddings may not accurately retain the exact semantics.

Program analysis approaches generally execute static/dynamic analysis to extract information (e.g., data/control dependencies) from the binaries and then quantify similarity based on certain defined

* Corresponding author at: Swinburne University of Technology, 1 John Street, Melbourne, VIC, 3122, Australia.
E-mail address: zianliu@swin.edu.au (Z. Liu).

https://doi.org/10.1016/j.neunet.2026.109800
Received 2 June 2025; Received in revised form 17 July 2026; Accepted 23 October 2026
Available online 28 October 2026
0893-6080/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
Comparison of our work with others.

Method	Approaches	Assembly syntactic Embedding	Training required
Asm2Vec (Ding et al., 2019)	Unsupervised neural network	✓	✓
Gemini (Xu et al., 2017a)	Supervised neural network	✓	✓
Palmtree (Li et al., 2021)	Supervised neural network	✓	✓
JTrans (Wang et al., 2022)	Unsupervised neural network	✓	✓
UniASM (Gu et al., 2025)	Unsupervised neural network	✓	✓
SemDiff (Ours)	Simple rule	✗	✗

rules (Alrabaee et al., 2015, 2018; Baker et al., 1999; Bourquin et al., 2013; David & Yahav, 2014; Farhadi et al., 2014; Hu et al., 2009; Huang et al., 2017; Kang et al., 2012; Lindorfer et al., 2012; Luo et al., 2014; Ming et al., 2017; Ng & Prakash, 2013; Santos et al., 2010; Wang & Wu, 2017; Xu et al., 2017b). However, static function-level processing approaches match a sequence of blocks or instructions, which defines each basic block or each instruction within a function as the smallest unit of similarity quantification. Considering the case of using different compiling optimization to proceed with the same source code, the basic block of each function will become totally different through block splitting, instructions can be different due to instruction substitution. Therefore, block or instruction comparison is not the ideal approach to handle binary similarity comparison (Ren et al., 2021).

To resolve the above limitations, we propose and implement a novel semantic-aware approach, SemDiff, to compare arbitrary binary codes without considering which compilers and optimization levels the developers utilized. Specifically, SemDiff consists of two modules, *Graph Generation* and *Graph Diffing*. In graph generation, SemDiff takes as input of a pair of binary functions and then constructs a key-semantics graph for each function by 1) identifying the key instructions that reflect major function behaviors (e.g., invoking functions, assigning value); 2) applying symbolic execution to extract symbolic expressions of each instruction and instruction dependencies; 3) translating symbolic expressions into self-defined key expressions and using directed edges to connect all the correlated instructions. After having a pair of key-semantic graphs of the two binary functions, graph diffing takes the graphs as input and leverages topological sort to serialize both graphs into two sequences. It then tokenizes each sequence and executes locality-sensitive hash (LSH)-based comparison to calculate an LSH value for each key-semantics graph. Two graphs (i.e., binary functions) are regarded as similar if their LSH values are similar. Note that the key instructions are empirically effective heuristic proposed by us to capture as much code semantics as possible.

We evaluated the binary similarity performance of SemDiff using nine libraries: openssl, libtomcrypt, coreutils, ImageMagick, libgmp, curl, sqlite3, zlib and Puttygen. We compared SemDiff with five state-of-the-art tools: Asm2Vec (Ding et al., 2019), Gemini (Xu et al., 2017a), Palmtree (Li et al., 2021), JTrans (Wang et al., 2022), and UniASM (Gu et al., 2025). The results demonstrated that SemDiff on average outperforms baseline tools no matter whether the source code is compiled by the same compiler (with different optimization levels) or the same optimization levels (with different compilers). The out-performance of SemDiff is because of the semantic-preserving key expressions and effective LSH-based graph diffing. In addition, we applied SemDiff to conduct vulnerability detection and library version check and found that SemDiff could significantly improve the performance of such similarity-dependent detection (see Table 1).

Summary of Contributions:

We summarize our major contributions as follows:

- We proposed a novel semantic-aware approach for binary similarity detection. We abstract binary code by selecting the key instructions only and then executing symbolic execution to extract the instruction correlations for further analysis. The approach efficiently simplifies binary code by refining exact semantic information.

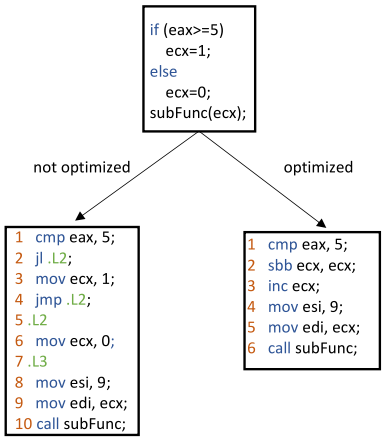


Fig. 1. Due to compiling optimization, a binary snippet on the left is obviously different in syntax and code structure from the one on the right, although they are from the same source code (on the top).

- We proposed an approach to translate the instruction summary into a graph. To achieve an accurate comparison, we propose an LSH-based approach to convert a graph into a sequence for the final similarity calculation. Such an approach could be applied to various scenarios (e.g., vulnerability searching, and malware detection). The comparison rules are very simple and our method does not require training.
- We assessed the performance of SemDiff by using 2106 binaries (considering various compilers, compiling options, source code versions, and obfuscations) in the 52 packages. and also compared the detection results with the state-of-the-art tools. The results demonstrated that SemDiff not only outperformed in binary similarity detection but also can be utilized to assist the other tools that require binary comparison. The results show that SemDiff achieves higher accuracy than GNN-based methods.
- We currently published our tool, SemDiff, and the experimental data to our repository https://anonymous.4open.science/r/SemDiff4BinaryDetection-F12C/README.md for reviewers to check and test. After this paper is accepted, the tool and dataset will be published.

Note that binary code in this paper represents the executable binary (e.g., ELF,/PE file) compiled by compilers.

2. Background

2.1. Motivation

We demonstrate a code compilation example in Fig. 1 to represent the binary diversity and how a compiling optimization changes a control-flow-graph (CFG). Specifically, a non-optimized binary snippet (on the left) has a conditional branch (`jle .L2`) with two destinations, i.e., `mov ecx, 1` and `mov ecx, 0`. An optimized snippet (on the right) substitutes the three instructions with `sbb`. Although both snippets are from the same source code, they have different syntax and structures.

The performance of supervised learning heavily depends on the training data. Therefore, supervised learning approaches (Ding et al., 2016; Feng et al., 2016; Gao et al., 2018; Xu et al., 2017a) will be ineffective for binaries compiled with unseen optimizations or compilers. Unsupervised learning automatically learns instruction embeddings from contexts. Therefore, if a non-trivial part of the context is different due to optimizations or different compilers, this approach (Ding et al., 2019; Duan et al., 2020) will be less accurate. Some program-analysis-based methods compare similarity at the granularity of basic blocks. However, as in the example, the blocks are merged after optimization. Thus methods based on basic-block level (Chandramohan et al., 2016; Flake, 2004; Luo et al., 2014) comparison can be ineffective for optimizations. Some program-analysis-based methods utilize Longest Common Sequence (LCS) to align two sequences of instructions or blocks from the two functions. However, optimization as shown in the example can change the instruction or basic block or their ordering, thus posing challenges to sequence aligning-based approaches (Luo et al., 2014; Ming et al., 2017; Wang & Wu, 2017).

2.2. Preliminary

In this section, we first present our goal and then give our assumptions of SemDiff.

Our goal: Detecting binary similarity can be conducted at different granularity, i.e., basic block, function, or inter-procedure CFG. A basic-block level quantifies the similarity between two given basic blocks. A function level quantifies the similarity between two given functions. An inter-procedure CFG quantifies two graphs of basic blocks connected across multiple functions following the control flow of a binary. Given that it is critical to find vulnerabilities in similar functions in the real world (Chandramohan et al., 2016; Feng et al., 2016; Gao et al., 2018), our goal is to effectively detect binary similarity at a function level, the same as Ding et al. (2016, 2019), Dullien (2018), Flake (2004), Luo et al. (2014). As mentioned in Section 1, binary similarity detection includes machine-learning and program-analysis approaches. In this paper, we use the program-analysis approach to compare binary similarity.

Our Assumptions:

Similar to previous works (Ding et al., 2019; Duan et al., 2020; Ming et al., 2017; Zuo et al., 2022), we make three assumptions that are practical in the real world. First, binaries are stripped without debugging information, as a stripped binary is released as a software product in the real world to protect its intellectual property. Second, binaries can be obfuscated, which is often used to make real binary code difficult to understand. Third, binaries are assumed to be unpacked, as binary unpacking is an orthogonal problem and can be solved by prior works (Cheng et al., 2018; Roundy & Miller, 2013).

2.3. Challenge

To compute the similarity of the two binaries precisely, the following challenges need to be resolved.

Challenge I: Extract Core Semantics. The syntax of the binary code varies when compiling the same source code with different compilers or optimizations. Extracting equivalent semantics from those two syntactically different programs is challenging. Existing solutions typically compare symbolic expressions after translating the binary into higher-level Intermediate Representations (IR) (Chandramohan et al., 2016; Luo et al., 2014). However, these IRs do not simplify the binary code. On the contrary, their grammar makes IR even more complex than binary code because they tend to translate one binary instruction to multiple IR instructions. Moreover, even translating to IR, IR still contains unmatched variables due to different compilers or optimizations. Therefore, comparing symbolic expressions of IR still cannot accurately identify semantic equivalence. Also, each binary instruction corresponds to at least one IR instruction. Therefore, a representation of the binary code that can both preserve semantics and simplify binary code is required.

Challenge II: Symbolic Formula Too Long. Instead of processing simple text-based analysis, many approaches (Ding et al., 2019; Gao et al., 2018; Xu et al., 2017a; Zuo et al., 2022) study the code similarities through semantic graph comparison. They extract graphs (e.g., control flow graphs, call graphs) and convert the graphs into vectors. Through machine learning algorithms, the binary code snippets with similar vectors will be identified. However, these approaches are not applicable. Specifically, those approaches regard each binary instruction as a vector of a fixed size, as different binary instruction generally are short (e.g., one operator with one or two operands) and has similar lengths. In reality, such a fixed size requirement is not applicable to our case, as our designed key expression might contain the semantics of multiple lines of instructions, thus can have extremely long expression, and the expression length can vary enormously. Instead, we adopt a locality-Sensitive hash (LSH) based approach to compute a hash value for all the nodes in the graph and further compare the hash values to calculate the similarity. LSH algorithm is able to represent high-dimensional data with low-dimensional data while preserving the relative distance among data.

Challenge III: Inefficient and Inaccurate Sequence Comparison To compare the function similarity, many works adopt a divide-and-conquer algorithm: longest common sequence (LCS) (Luo et al., 2014; Ming et al., 2017; Wang & Wu, 2017). In LCS, each function is treated as a sequence of instructions or blocks. At the lowest level, it compares one instruction with another instruction or one block with another block. However, the comparison strategy at the lowest level is to be defined by us (i.e., how to compare the similarity between two instructions or two blocks). Therefore, human experts need to inspect the binary code and conclude rules for comparison. The accuracy heavily depends on the quality of the set comparison rules. Also, it is time-consuming for human to propose rules, making this approach unscalable. Moreover, LCS is NP-hard, the running time cost is significant. Nonetheless, these approaches cannot process the graph of binary codes efficiently.

2.4. Solution

Regarding the challenges to binary comparison, we propose the following approaches.

Respond to challenge 1, we propose a symbolic-based binary translation approach to abstract key instructions from each binary and generate semantic-aware representatives for further comparison. After observing the binary codes generated from the same high-level source code, we found that instructions can be classified into *key instructions* and *non-key instructions*. In particular, key instructions represent the major function executions and parameter value transmission and non-key instructions are the instructions for preprocessing purposes such as address computation.

Through our manual inspection, we classified the key instructions into four types, *calling behavior*, *comparing manner*, *indirect branch*, and *memory store*.

- **Calling behavior.** It represents a calling instruction that takes operands operated by previous instructions as function arguments.
- **Comparing manner.** It is an instruction with operands of comparing objectives. The instruction with the operator such as `cmp`, `test` will affect which subsequent branches to execute at a joint point.
- **Indirect branch.** It represents an instruction with an operand of a target address (e.g., `jmp eax`).
- **Memory store.** It is an instruction with the operand that stores values or memory addresses (e.g., `mov [edx], ebx`).

Remark 1. Note that, since we use symbolic execution, in the case of indirect branches, the value of eax will be abstracted as a symbolic expression. For example, at some point in the assembly code, `eax` might be represented as `(VAR0 + 1000) & 2`. The exact symbolic expression depends on how the value was computed in the preceding instructions.

Table 2
Key instructions and their corresponding key expressions.

Key Instruction	Key Expression
calling behavior	$RET_FuncAddr(exp_1, \ldots, exp_n)$
comparing manner	$exp_1\ cmp\ exp_2$
indirect branch	$branch\ exp$
memory store	$[exp_1] = exp_2$

Additionally, since we use symbolic execution, this example instruction of the memory store operation might be translated to `[VAR1 - 1000] = (VAR3 * 3) + 2`. This is because the symbolic expression for `edx` was calculated as `VAR1 - 1000`, while the symbolic value for `ebx` was computed as `(VAR3 * 3) + 2` based on the previous instructions. As a result, the impact of compiler optimizations is mitigated.

We select these four key instructions because theoretically, these four instruction categories collectively as an empirically effective heuristic to capture the as much program semantics along data-flow (through memory writes, calls), control-flow (through compares and indirect jumps), and inter-procedural dimensions (through calls). Other instruction types contribute to operand computation but do not alter the program's observable behavior beyond these semantic boundaries. Also, in the experiment, the empirical study shows that those four key instructions covers more than 94% code coverage on average.

Therefore, we further define four types of *key expressions* corresponding to the types of key instructions, shown in Table 2. For the key expression of a calling behavior, RET denotes a `call` instruction. $FuncAddr$ is the starting address of a function, and exp_i ($i \in \{1, \ldots, n\}$) is the symbolic expression of the function arguments. For the key expression of a comparing manner, cmp denotes a comparing instruction. exp_1 and exp_2 are two operands used for comparison. For the key expression of an indirect branch, $branch$ denotes a branching instruction and exp refers to the symbolic expression of the branch destination. For the key expression of a memory store, exp_2 is written into the memory address denoted by exp_1. According to the defined key expressions, we first symbolically execute the binary to derive each instruction's each operand's symbolic expression and then translate each marked key instruction into specific key expressions (see Section 3.2 for details).

To address challenges 2 to 3, we utilize the LSH algorithm. Specifically, given the abstracted binary with key expressions, we raise an LSH-hash-based comparison to compute the similarity of the two abstracted assemblies. Specifically, we first build a *key-semantics graph*, which summary the major behavior of each function. Referring to node correlations demonstrated in the graph, we topologically sort all nodes and use LSH hashing to hash each graph into an LSH value. By comparing the LSH values of the two graphs, we can finally speculate the similarity between the two graphs.

3. SemDiff

3.1. Overview

For two binary candidates, e.g., `BinA` and `BinB`, each has a set of functions, that is, $\{FuncA_1, \ldots, FuncA_n\}$ and $\{FuncB_1, \ldots, FuncB_m\}$. To detect binary similarity at function level, we select a pair of $FuncA_i$ ($i \in \{1, \ldots, n\}$) and $FuncB_j$ ($j \in \{1, \ldots, m\}$) from `BinA` and `BinB`, and feed them into SemDiff for similarity quantification. SemDiff consists of two modules, i.e., graph generation and graph diffing, which work as follows:

- **Graph Generation**. It contains three major steps. First, we leverage customized symbolic execution to extract symbolic expressions of key instructions from a given function. Second, we translate the extracted symbolic expressions into key expressions. Last, we generate a graph preserving the key semantics of a function by connecting the translated key expression to the function's control flow (see Fig. 2).
- **Graph Diffing**. It has four major steps. First, we serialize a key-semantics graph into a sequence of key expressions by topological sort. Second, we tokenize each key expression to produce a list of token sequences. Third, we concatenate all the token sequences for all the key expressions and use the locality-sensitive hash (LSH) to hash the concatenated tokens and generate an LSH hash value for one function. Last, we diff two given functions by quantifying the `Jaccard` similarity between two generated LSH hash values.

By doing so, a similarity score will be computed for the selected pair of functions. For each function in `BinA`, we do a 1-to-n compare with all the functions in `BinB` and pick a pair of functions that has the highest score as the most similar one. If the picked pair has the same function name, it means that we detect the correct pair.

Note that, compared to GNN-based methods, SemDiff does not require collecting a significant amount of training data or performing time-consuming pretraining. This alleviates the challenge of obtaining abundant, high-quality assembly code for training in real-world applications.

3.2. Graph generation

3.2.1. Key-instruction symbolic expression extraction

To efficiently extract all symbolic expressions of key instructions from a given function, we customize symbolic execution by proposing two techniques. First, we symbolically execute a function to traverse all its instructions. Second, we symbolically execute a loop in a lightweight way rather than repeatedly executing the loop till the loop condition is not satisfied.

Algorithm 1: Complete instruction traversal.

```
1  Function execute_next_node(Node):
2      if Node has not been executed before then
3          symbolic_execution(Node)
           // symbolically execute the instruction and update
              relevant records of symbolic expressions.
4          foreach j ∈ Node.children do
5              execute_next_node(j)
6          end
7      end
8      else if Node forms a loop then
9          loop = extract_loop(Node)
10         lightweight_loop_processing(loop)
11     end
```

Traversing All Instructions in A Function: For a given function in a binary, we perform a complete instruction traversal as shown in Algorithm 1. The input for the algorithm is a function's first instruction. The function is regarded as a control-flow graph where a node denotes an instruction and children of the node are subsequent instructions conforming to the function's control flow. At the beginning of the algorithm, we also need to provide symbolic values to the function's input arguments. Particularly, we assign symbolic values VAR_i ($i \in N$) to relevant registers. The order for assigning values for the registers is based on the x86-64 calling conventions. For example, on x64 Linux, register `rdi,rsi, rdx, rcx` represents arguments 1 to 4 of the function.

The Algorithm 1 implements a depth-first searching function, i.e., *execute_next_node*. which achieves a complete instruction coverage rather than a complete code path to avoid the serious path-explosion problem. Specifically, we first check whether a node has been executed before (Line 2). If no, we symbolically execute the node (Line 3). As an instruction in a node can have one or more operands, each of its operands will produce a symbolic expression after the symbolic execution. Thus, we maintain a record of symbolic expressions for each operand in each instruction. If the data to be referenced is resolvable (e.g., `mov esi,`

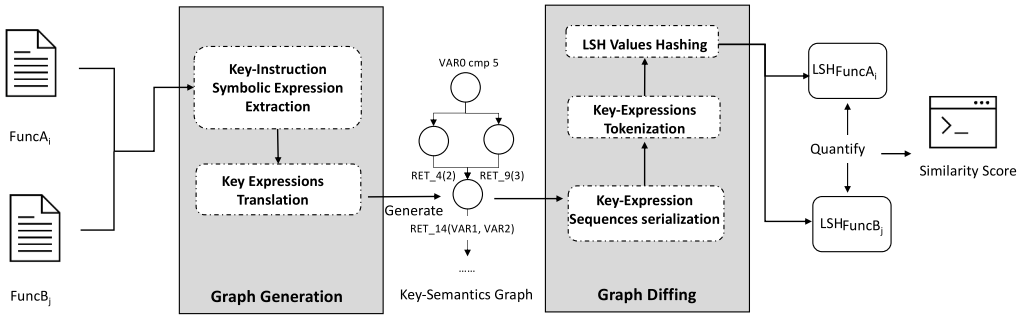


Fig. 2. SemDiff Overview. SemDiff consists of two modules, i.e., graph generation and graph diffing. (LSH is short for locality-sensitive Hash.)

`address` where `address` points to a string of `''Rtmin''`), we use its resolved value (`''Rtmin''`) to continue the symbolic execution. If data is unresolvable (e.g., `mov edi, cs:bio_err` where `cs:bio_err` is unknown), we assign unused VAR_i ($i \in N$) to represent unknown values. Remark: IDA Pro-often recognizes when an immediate value or constant represents a memory address and renames it to a variable, such as a variable name, as shown in the example above. In cases where IDA Pro-cannot effectively recognize that a constant is an address—due to address hiding (e.g., **lea + add** instructions) or obfuscation—our method includes an additional step to compare whether the calculated constant falls within the address space. If it does, SemDiff treats it as an address by default. As a result, the constant value is not used for comparison; instead, a symbolic expression, such as **VAR3**, is used for comparison.

If the node has been executed before, we check whether the node is the start of a loop (Line 8). If yes, we process the loop in a lightweight way, i.e., *lightweight_loop_processing*() shown in Line 10 and discussed in *Lightweight Loop Processing* later. If the node has been executed before but does not form a loop, we simply return (Line 11) to avoid repeated node execution.

Lightweight Loop Processing: In the symbolic execution above, it is inefficient to execute a loop as the loop can be repeated many times or even infinite. Instead, we propose a lightweight approach. Considering that a loop updates one or more variables each time (e.g., adding or subtracting a counter value), we call such a variable a *loop counter*. When we encounter a loop, we execute the loop only two times. If there are branches within the loop, we randomly select one branch at the first time of loop execution. In the second time loop execution, we follow the same path. For example, in Fig. 3, there is more than one branch in the loop. In the first time of execution, we randomly select a path $.L1 - .L3 - .L4$. We execute $.L1 - .L3 - .L4$ and their symbolic expressions are denoted in Fig. 3 after each `1st:` symbol. If there is more than one operand, the symbolic expressions of the operands are separated by a comma. For example, `1st: 3,3` at line 5 in $L3$ means that, after the symbolic execution, the first operand's symbolic expression is 3, and the second operand's symbolic expression is also 3. In the second time execution, we follow the same path. Their symbolic expressions are denoted after each `2nd:` symbol. We then compare each operand's symbolic expression after the first and second execution to detect operands with changed symbolic expressions. We add a symbol `ITER` as the prefix to it, meaning it is a loop counter. For example, in Fig. 3, we identify `eax` in lines 7 and 8 as loop counter because the first time the symbolic expression of `eax` is *VAR0* and the second time the symbolic expression of `eax` is `VAR0+1`. This means this variable increase by 1 in each iteration. Therefore, we change `eax`'s symbolic expression to `ITER(VAR0)`.

3.2.2. Key expressions translation

As symbolic Expressions of Key Instructions have been extracted, they can be complicated and still have many syntactic differences due to compiling techniques such as data encoding (Nagra & Collberg, 2009) and Mixed-Boolean-Arithmetic (Zhou et al., 2007). As such, we further translate them into key expressions in two steps. First, we use

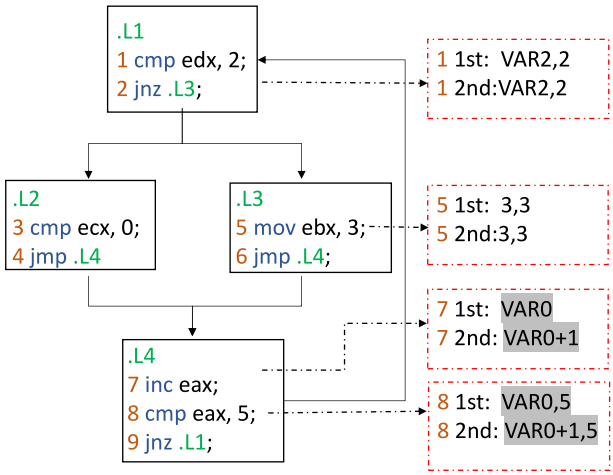


Fig. 3. An example of a loop.

expression-synthesizing techniques (David et al., 2020; Kan et al., 2019; Saidi et al., 2010) to synthesize a symbolic expression into a simplified one, e.g., $x \vee y - x \wedge y \Rightarrow x \bigoplus y$. This technique can effectively transform long and complex symbolic expressions into simpler and shorter expressions. Second, we translate each key instruction to key expression according to the rules as shown in Table 2.

3.2.3. Generating a key-semantics graph

Key instructions in a function have been turned into key expressions. However, until now, the key instructions are still an unsorted list, as highlighted in red rectangles in Fig. 4. Now we connect the key expressions based on a function's control flow to produce a key-semantics graph. Each node in the key-semantic graph represents a key instruction, represented as a vertex $V = \{V_1, \ldots, V_n\}$ in the key-semantics graph. The vertex attribute $Attr_i$ ($i \in \{1, \ldots, n\}$) is its key expression. The edges $E = \{(i, j) \mid i, j \in V^2\}$ in the graph represent the control-flow among key instructions.

For example, Fig. 4 shows how graph generation processes a pair of binary functions compiled from the same source code with and without optimization. Specifically, it first extracts symbolic expressions of key instructions from a given binary function and then translates the extracted expressions into key expressions. Last, it connects the key expressions based on the function's control flow to generate a key-semantics graph.

3.3. Graph diffing

To quantify the similarity between the two given graphs, existing approaches vectorize the attribute of each node. Thus, this requires the attributes must be short symbols. However, the attributes of our key-semantic graph can be long expressions, making existing approaches inapplicable.

Table 3
Key expression tokenization.

Key-Instruction Type	Key Expression	A List of Token Sequences
calling behavior	$RET_FuncAddr(exp_1, \dots, exp_n)$	$RET_(exp_1_token_1), \dots, RET_(exp_1_token_n), \dots;$ $RET_(exp_n_token_1), \dots, RET_(exp_n_token_n)$
comparing manner	$exp_1\ cmp\ exp_2$	$cmp\ exp_1_token_1, \dots, cmp\ exp_1_token_n, \dots;$ $cmp\ exp_2_token1, \dots, cmp\ exp_2_token_n$
indirect branch	$branch\ exp$	$branch\ exp_token_1, \dots, branch\ exp_token_n$
memory store	$[exp_1] = exp_2$	$[exp_1_token_1] =, \dots [exp_1_token_n] =;$ $= exp_2_token_1, \dots, = exp_2_token_n$

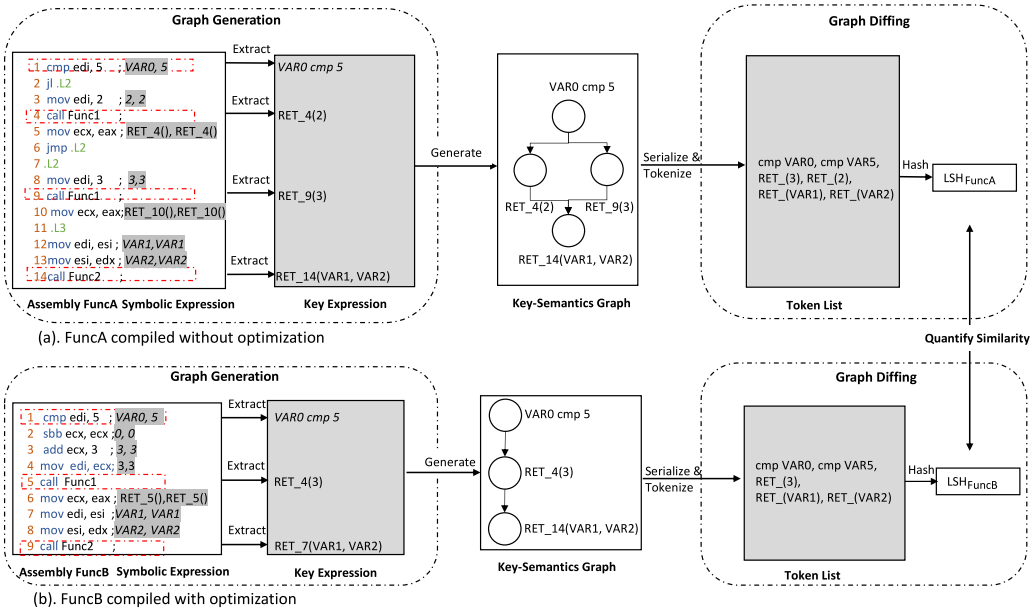


Fig. 4. An example of (key-semantics) graph generation and graph diffing. FuncA and FuncB are compiled from the same source code without and with optimizations.

Inspired by Lin et al. (2019), Yamaguchi et al. (2012) where source codes are transformed into a sequence of instructions to traverse an Abstract Syntax Tree in a linear order, we address the problem in three steps. 1) We serialize a key-semantics graph into a sequence of nodes by *topological sort*. 2) We tokenize the key expressions (i.e., each node attribute in the serialized graph). 3) We concatenate the tokenized key expressions and apply *locality-sensitive hash* (LSH) to produce an LSH value for similarity quantification.

3.3.1. Key-expression sequences serialization

The technique of topological sort can sort all the nodes $V = \{V_1, \dots, V_n\}$ of a directed graph $G = (V, E)$ in a linear order and thus G satisfies the following property: every directed edge $\{(V_u, V_v) \mid V_u, V_v \in V^2\}$ is forward, i.e., V_u comes before V_v. More specifically, topological sort can keep the structural and geometrical relations among the nodes. Similar graphs can result in similar topological sequences.

However, it is possible that a loop in the function contains multiple key instructions. Therefore, graph generation step can produce loops in key-semantics graph. And topological sort works only for a directed graph that has no loop, which cannot be directly applied to a key-semantics graph that can contain loops. To address this issue, we first make a key-semantics graph loop-free by removing the last edge in the flow of a loop and then adding a symbol of $WHILE$ into the starting node of the loop, meaning it is the beginning of the loop. As shown in Fig. 3, $.L4$ to $.L1$ is the last edge of the flow and is removed. The nodes in the loop body are retained.

3.3.2. Key-expressions tokenization

For each node attribute in the serialized graph, we split its key expression into operands and operators, constituting a sequence of tokens, e.g., $X + 3 - Y + 7 * Z$ is spit and tokenized into a sequence of $X, 3, -, 7, *, Z$. If a key expression has brackets or parentheses, we retain the brackets or parentheses for each token when splitting it, e.g., parts of two key expressions are $[X + [Y + Z - 3] * 2]$ and $(X + (Y * 6 + (Z - 3 + K)))$. They are tokenized as: $[X], [[Y]], [[Z]], [[-]], [[3]], [*], [2]$ and $(X), ((Y)), ((*)), ((6)), (((Z))), (((-))), (((3)))(((K)))$. We note that the plus symbol (+) in a key expression is omitted when it is split, as we observe that a tokenized plus symbol dominates a token sequence, which makes two different token sequences have a high similarity score. To associate a token with its corresponding key-expression type, we prefix a token with a symbol as shown in Table 3.

- For a calling behavior, each token is prefixed with $RET_()$, denoting that it comes from a calling instruction, e.g., a key expression is $RET_(3, VAR0 + 4)$ becomes $RET_(3), RET_(VAR0), RET(4)$.
- For a comparing manner, each token is prefixed with cmp, e.g., a key expression $4\ cmp\ [VAR1 + 18]$ turn into $cmp\ 4, cmp\ [VAR1], cmp\ [18]$.
- For an indirect branch, each token is prefixed with $branch$, e.g., $branch\ [[VAR2 + 10] + 16]$ turns into $branch\ [[VAR2]], branch\ [[10]], branch\ [16]$.
- For a memory store, tokens on the left of equal sign ends with $=$, tokens on the right of the equal sign prefixed with $=$, e.g., $[VAR2 + 18] = (VAR1 + 10) * 3$ becomes $[VAR2] =, [18] =, = (VAR1), = (10), = *, = 3$.

3.3.3. LSH values hashing and quantifying similarity

The LSH approach is effective for nearest neighbor search in high-dimensional spaces as LSH can represent high-dimensional data in a low-dimension format while preserving the relative distance between data. Particularly, this approach hashes data objects into buckets so that similar objects will be hashed into the same bucket with a high probability. As such, we utilize LSH to generate an LSH value for a serialized and tokenized key-semantics graph. Specifically, we first concatenate all the token sequences from all the key expressions in a serialized key-semantics graph. We then generate an LSH value for the concatenated token sequence to represent one function. With two LSH values from two given functions, we use the *Jaccard similarity* to quantify the similarity between the two functions, as the Jaccard similarity shown below is commonly used to measure similarity or distance for two given datasets.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

where A and B are two datasets, and the number of the common elements in them is divided by their total number.

Fig. 4 shows an example of how graph diffing operates. It first serializes two given key-semantics graphs and then tokenizes the key expressions, which are concatenated to generate a token list. Last, it computes LSH values for token lists from each binary function and quantifies their similarity.

Remark: In our implementation, we use Python's **networkx** library's topological sort API, along with **datasketch's Minhash**, to serialize the key-semantics graph into a token list and hash it.

Source Lines of Code: Our implementation for the first module contains 12,527 `C++` SLOC and it serves as a plugin to `IDA Pro`. The second module has 2913 python SLOC. We use APIs provided by `msynth`[1] as a python package to synthesize symbolic expressions of key instructions.

4. Evaluation

We evaluated the effectiveness of SemDiff answering the following research questions (**RQs**):

- **RQ1.** What is the accuracy of the generated key expressions?
- **RQ2.** Can SemDiff detect similarity across different compiling optimizations?
- **RQ3.** Can SemDiff detect similarity across different compilers?
- **RQ4.** Can SemDiff detect similarity across different obfuscations?
- **RQ5.** Can SemDiff be applied to cross-version similarity detection?
- **RQ6.** How does SemDiff perform in vulnerability detection?
- **RQ7.** Can the proposed key instructions effectively capture the program semantics?
- **RQ8.** Does assigning different weights to the various key instructions affect the accuracy?

We have grouped those research questions into groups: (1) Semantic representation accuracy: RQ1, (2) Robustness against compilation differences: RQ2-4, (3) Real-world applications: RQ 5-6, (4) Design effectiveness: RQ7-8.

4.1. Experiment setup

Dataset We conducted experiments on nine popular open-source projects, i.e., *openssl-3.3.0*, *libtomcrypt-1.18.2*, *coreutils-8.32*, *ImageMagick-7.1.010*, *libgmp-6.2.1*, *curl-7.80*, *sqlite3-3.37.0*, *zlib-1.2.11* and *Puttygen-0.74*. From these libraries, we selected 13 programs (listed in Table 5) that are widely used by vendors and the other researchers (Ding et al., 2019; Mengin & Rossi, 2021; Wang & Wu, 2017). The source code of these programs consists of 52 project packages. The compiled binary code include 2106 binaries, with more than 416,000 assembly functions due to different compilers, compiling options, source code versions, and obfuscations.

Table 4
Statistics of the analyzed functions.We randomly selected 200 functions in total from various projects with different optimization levels.

Project	Functions (#)	Project	Functions (#)
Coreutils-O0	25	Curl-O1	30
Coreutils-O3	20	libgmp-Os	31
Coreutils-Os	27	libmagickcore-o1	24
libtomcrypt-O1	23	libmagickcore-o2	20

Environment We set up a machine with an Intel NUC8i5BEH (Intel processor i5-8259U with 16 GB memory). Since the experiment might involve deep learning, we also use an accelerator cluster of HPC systems with 456 NVidia Tesla P100, 114 Dual Xeon 14-core E5-2690 v4 compute nodes and 256 GB memory.

Experiment We compare SemDiff with five state-of-the-art tools of binary similarity detection (i.e., *JTrans* (Wang et al., 2022), *UniASM* (Gu et al., 2025), *Asm2Vec* (Ding et al., 2019), *Gemini* (Xu et al., 2017a), and *Palmtree* (Li et al., 2021)). We trained *JTrans* (Wang et al., 2022), *UniASM* (Gu et al., 2025), *Gemini* and *Palmtree* on the HPC, and validate all the tools on Intel NUC. *JTrans*, *functionsimsearch*, *Gemini*, and *Palmtree* are machine learning based approaches. Hence, we leverage a total of 7000 assembly functions from four programs (i.e., *busybox*, *coreutils*, *libgmp*, and *libMagickCore*) compiled with different optimization levels for model training. Aligned with existing works (Ding et al., 2019; Eschweiler et al., 2016; Feng et al., 2016), we consider functions with at least five basic blocks as functions with less than 5 blocks are less likely to contain bugs, and thus are of less interest for similarity detection.

Similarity Measurement Given a pair of binary codes (i.e., `BinA` and `BinB`) compiled by different optimization levels, we compute the similarity score as below. First, we select a function from `BinA` and execute a binary similarity detection tool to acquire a similarity score between the selected function and each function in `BinB`. The pair of functions with the largest score is regarded as the most similar function. Then we enable the debugging information of the function based on the function name to check whether functions in `BinB` and `BinA` are the same. To quantify the similarity, we use the same metrics as the previous works (Ding et al., 2019; Ren et al., 2021). That is precision at position 1 (precision@1), which captures the ratio of binary functions from `BinA` that correctly find the function with the same name in `BinB` at position 1. Precision@1 is equal to Recall at Position 1 (Recall@1) in this case.

Please note that in the following sections, we use the aforementioned program versions for evaluation. For Section 4.4.1, we use the aforementioned program version as the baseline and compared them with other versions in terms of their binary similarity, e.g., curl-3.3.0 is against curl-7.43, curl-7.65 and curl-7.72.

4.2. RQ1: Semantic representation accuracy – correctness of key expressions

To justify the correctness of the translated key expressions, we randomly selected 200 functions from the dataset described in Section 4.1 and asked three experienced programmers to label them manually. The 200 functions are shown in Table 4. It demonstrated that SemDiff can correctly translate 85% of instructions. By manually inspecting the incorrect cases, we observed that SemDiff cannot recognize (currently not supported) some x64 mnemonics' variants (e.g., `movzx`) that are less frequently used in binary codes. As SemDiff is built on top of IDA pro to resolve string variable names into contents of the string, IDA pro might mistakenly resolve the strings. For example, IDA pro may consider constant value as a memory address and resolve the content in that memory address.

[1] https://github.com/mrphrazer/msynth

Table 5
Libraries compiled by GCC 12.4 with different optimization levels.

Program	JTrans	UniASM	Asm2Vec	Gemini	Palmtree	GMN	Trex	SemDiff
	O1-00 O3-00 Os-O0	O1-00 O3-00 Os-00	O1-00 O3-00 Os-O0	O1-00 O3-00 Os-00	O1-00 O3-00 Os-O0	O1-00 O3-00 Os-O0	O1-00 O3-00 Os-O0	O1-00 O3-00 Os-O0
openssl	(0.723,0.637,0.625)	(0.678,0.679,0.623)	(0.803,0.701,0.712)	(0.315,0.298,0.231)	(0.369,0.326,0.347)	(0.479,0.489,0.497)	(0.497,0.480,0.488)	(**0.819**,**0.724**,**0.735**)
libtomcrypt	(0.683,0.679,0.668)	(0.716,0.733,0.69)	(0.669,0.617,0.715)	(0.086,0.045,0.078)	(0.178,0.183,0.215)	(0.606,0.593,0.585)	(0.596,0.582,0.581)	(**0.817**,**0.736**,**0.753**)
coreutils	(0.586,0.457,0.406)	(0.579,0.457,**0.689**)	(0.577,**0.516**,0.589)	(0.266,0.279,0.273)	(0.387,0.275,0.342)	(0.517,0.467,0.53)	(**0.587**,0.416,0.419)	(0.562,0.486,0.637)
libMagickCore	(0.737,0.653,0.674)	(0.709,0.719,0.647)	(0.679,0.705,**0.798**)	(0.189,0.114,0.198)	(0.237,0.197,0.184)	(0.602,0.603,0.611)	(0.598,0.597,0.601)	(**0.749**,**0.759**,0.619)
libMagickWand	(0.849,0.816,0.793)	(0.904,0.816,0.787)	(0.440,0.767,0.807)	(0.096,0.059,0.067)	(0.196,0.116,0.153)	(0.655,0.632,0.601)	(0.639,0.602,0.610)	(**0.919**,**0.869**,**0.896**)
libgmp	(0.786,0.705,0.658)	(0.790,0.748,0.738)	(0.791,**0.889**,0.715)	(0.453,0.517,0.427)	(0.668,0.762,0.526)	(0.562,0.612,0.559)	(0.591,0.615,0.570)	(**0.796**,0.816,**0.775**)
curl	(0.838,0.706,0.769)	(0.856,0.716,**0.784**)	(0.775,0.628,0.681)	(0.360,0.349,0.358)	(0.594,0.375,0.574)	(0.530,0.409,0.512)	(0.532,0.446,0.502)	(**0.896**,**0.727**,0.773)
sqlite3	(**0.783**,0.675,**0.685**)	(0.768,0.694,0.626)	(0.756,0.517,0.630)	(0.276,0.260,0.184)	(0.217,0.174,0.285)	(0.519,0.403,0.522)	(0.523,**0.596**,0.527)	(0.716,0.575,0.669)
libz	(0.768,0.619,0.795)	(**0.775**,0.669,0.806)	(0.774,**0.767**,0.863)	(0.284,0.269,0.264)	(0.358,0.254,0.424)	(0.658,0.647,0.703)	(0.635,0.607,0.614)	(0.737,0.685,**0.889**)
plink	(0.659,**0.601**,0.663)	(0.675,0.594,0.634)	(0.625,0.593,0.645)	(0.356,0.316,0.309)	(0.367,0.254,0.376)	(0.578,0.516,0.502)	(0.525,0.501,0.497)	(**0.758**,0.559,**0.847**)
pscp	(0.686,0.584,0.674)	(0.706,**0.604**,0.684)	(0.676,0.578,0.656)	(0.306,0.286,0.263)	(0.397,0.286,0.296)	(0.568,0.497,0.474)	(0.532,0.482,0.480)	(**0.748**,0.585,**0.884**)
psftp	(0.623,0.567,0.627)	(0.674,0.523,0.664)	(0.675,**0.572**,0.598)	(0.262,0.207,0.241)	(0.374,0.289,0.383)	(0.503,0.413,0.524)	(0.533,0.468,0.517)	(**0.795**,0.547,**0.774**)
puttygen	(0.685,0.501,0.651)	(0.663,0.525,0.674)	(**0.698**,0.476,0.674)	(0.243,0.269,0.274)	(0.396,0.227,0.364)	(0.532,0.469,0.608)	(0.528,**0.593**,0.604)	(0.674,0.563,**0.773**)
Avg.	(0.723,0.630,0.668)	(0.730,0.652,0.695)	(0.687,0.640,0.698)	(0.268,0.251,0.243)	(0.364,0.286,0.343)	(0.562,0.519,0.556)	(0.562,0.537,0.539)	(0.768,0.663,0.771)

4.3. Robustness against compilation differences

4.3.1. RQ2: Cross-GCC-compiling-optimization-level similarity detection

GCC is one of the most widely used compilers in the real world and thus we choose it for cross-compiling-optimization-level evaluation. We conduct the experiment by setting five optimization levels, *O*0, *O*1, *O*2, *O*3, and *Os*, and apply SemDiff and the other five state-of-the-art tools to detect the similarity of the binary codes generated from the same source function, but compiled by using different optimization levels. In addition, we leverage the metrics of Normalized Compression Distance (NCD) score (Alshahwan et al., 2020; Borbely, 2015; Raff & Nicholas, 2017a,b; Ren et al., 2021) to quantify the syntactic similarity of the binary code pair. A higher NVD score represents the binary code pair looks more dissimilar.

The similarity detection results are listed in Table 5. Due to the page limitation, we only list the detection results of the most dissimilar pair (i.e., *O*3 vs *O*0) and the two other dissimilar pairs (i.e., *O*1 vs *O*0 and *Os* vs *O*0). The other results and the corresponding discussions are all listed on our website.[2] Remark: Note that -O2 is not included in the comparison because it largely overlaps with -O1 and -O3, and therefore is not expected to produce qualitatively different behavior than those already presented. In GCC, both -O3 and -Os have significant overlap with -O2. Table 7 provides a more detailed comparison of the different optimization levels. Each optimization level consists of a set of specific optimization flags. The similarity detection results illustrate that SemDiff averagely achieves a detection precision at 73%, but the other detection tools can only achieve a detection prediction from 18% to 74.9%. Although *JTrans, UniASM, Asm2Vec* slightly performs better than SemDiff in some cases, SemDiff could maintain a high detection performance even though the pair of binary codes look the most dissimilar.

4.3.2. RQ3: Cross-compiler similarity detection

Similarly, we conduct the experiment by using two different compilers (i.e., *GCC 12.4* and *CLANG 19.1.0*), but the same optimization level to assess the similarity detection performance of SemDiff. By checking the NVD score of the binary code pairs, we found that *GCC O*0 vs *CLANG O*0 and *GCC O*1 vs *Clang O*1 are the two most dissimilar binary code pairs. Therefore, we listed the similarity detection results in the paper (shown in Table 6). The rest results are listed on our website as well.

On average, SemDiff could achieve a detection precision at 81% when using the same optimization level, but different compilers. Also in some cases, *JTrans*, *UniASM*, and *Asm2Vec* performs slightly better than SemDiff.

For binaries compiled from the same source code using different optimization levels in Section 4.3.1 or different compilers in Section 4.3.2, their function numbers vary mainly due to the `inline` optimization, which inserts functions being called into the callee function. Also, they are likely to differ in almost all the binary functions as some instructions inside a function have syntactic differences but the same semantics. These differences will result in different function attributes such as the statistics of basic blocks, instructions, and mnemonics. Thus methods that rely on syntactic information (all except SemDiff) are less accurate. However, most key semantics of a function is still preserved in this case, making SemDiff more effective than other tools.

4.3.3. RQ 4: Different obfuscation options similarity detection

We also quantify the similarity detection performance by using different obfuscation options, that is two programs that are compiled by *CLANG 3.8.0* with *O*0 and *OLLVM* (Junod et al., 2015) with three different obfuscation options (i.e., SUB, BCF, and FLA). *OLLVM* with

[2] https://anonymous.4open.science/r/SemDiff4BinaryDetection-F12C/README.md

Table 6
Binary comparison result when using different compilers.

Program	JTrans O0-O0 O3-O3	UniASM O0-O0 O3-O3	Asm2Vec O0-O0 O3-O3	Gemini O0-O0 O3-O3	Palmtree O0-O0 O3-O3	GMN O0-O0 O3-O3	Trex O0-O0 O3-O3	SemDiff O0-O0 O3-O3
openssl	(0.732,0.852)	(0.762,0.801)	(0.77,0.887)	(0.375,0.353)	(0.175,0.278)	(0.675,0.634)	(0.564,0.532)	(**0.864**,**0.877**)
libtomcrypt	(0.652,0.763)	(0.625,0.763)	(0.663,0.762)	(0.152,0.146)	(0.164,0.160)	(0.565,0.542)	(0.601,0.612)	(**0.764**,**0.874**)
coreutils	(0.763,**0.862**)	(0.716,0.813)	(0.637,0.773)	(0.232,0.162)	(0.264,0.273)	(0.686,0.675)	(0.587,0.553)	(**0.775**,0.845)
libMagickCore	(0.643,0.765)	(0.674,0.753)	(0.664,0.835)	(0.258,0.207)	(0.286,0.264)	(0.509,0.516)	(0.755,**0.863**)	(**0.784**,0.847)
libMagickWand	(0.676,0.784)	(0.675,0.886)	(0.384,0.426)	(0.108,0.169)	(0.086,0.064)	(0.701,0.714)	(0.695,0.687)	(**0.873**,**0.906**)
libgmp	(0.557,0.593)	(0.523,0.696)	(0.497,0.596)	(0.267,0.273)	(0.137,0.286)	(0.387,0.512)	(0.398,0.601)	(**0.575**,**0.720**)
curl	(0.817,0.789)	(0.853,0.725)	(0.863,0.705)	(0.584,0.462)	(0.273,0.363)	(0.814,0.856)	(0.826,0.798)	(**0.938**,**0.912**)
sqlite3	(0.733,0.663)	(0.735,0.609)	(0.703,0.627)	(0.261,0.275)	(0.138,0.147)	(0.725,0.601)	(**0.775**,0.598)	(0.727,**0.716**)
libz	(0.746,0.683)	(0.728,0.797)	(0.895,0.738)	(0.586,0.438)	(0.389,0.338)	(0.854,0.789)	(0.725,0.617)	(**0.957**,**0.838**)
plink	(0.738,0.686)	(0.705,0.721)	(0.738,0.709)	(0.238,0.296)	(0.138,0.238)	(0.689,0.654)	(0.709,0.716)	(**0.894**,**0.838**)
pscp	(0.738,0.705)	(0.759,0.727)	(0.748,0.76)	(0.227,0.206)	(0.389,0.248)	(0.728,0.710)	(0.698,0.687)	(**0.720**,**0.826**)
psftp	(0.638,0.749)	(0.648,0.749)	(0.697,0.737)	(0.249,0.238)	(0.139,0.228)	(0.643,0.603)	(0.679,0.732)	(**0.859**,**0.838**)
puttygen	(0.785,0.648)	(0.738,0.726)	(0.747,0.705)	(0.305,0.206)	(0.227,0.249)	(0.717,0.616)	(0.690,0.704)	(**0.849**,**0.802**)
Avg.	(0.709,0.734)	(0.703,0.751)	(0.692,0.712)	(0.295,0.263)	(0.215,0.241)	(0.668,0.647)	(0.669,0.669)	(0.813,0.833)

Table 7
Comparison of different optimization levels.

Compared levels	Common flags	Extra Flags	Removed/Different
O0 vs O1	~0	+30~35 in O1	None
O1 vs O2	~30	+25 in O2	None
O2 vs O3	~60	+10~15 in O3	None
O2 vs Os	~45-50	+5~10	~10-15

Table 8
The similarity scores are computed by 6 tools for open-source programs of different versions compiled by GCC 5.4.0.

Program	Version	JTrans	UniASM	Asm2Vec	Gemini	Palmtree	SemDiff
openssl	1.0.2o vs. 3.0.0	0.522	0.463	0.428	0.284	0.382	**0.534**
	1.1.1i vs. 3.0.0	0.753	0.652	0.839	0.576	0.649	**0.918**
	1.1.1 vs. 3.0.0	0.667	0.676	0.813	0.568	0.648	**0.923**
coreutils	8.27 vs. 8.32	0.859	0.825	**0.895**	0.551	0.626	0.877
	8.29 vs. 8.32	0.856	0.761	**0.899**	0.576	0.671	0.878
	8.31 vs. 8.32	**0.932**	0.897	0.916	0.591	0.673	0.900
curl	7.43 vs. 7.80	0.697	0.598	0.800	0.438	0.603	**0.908**
	7.65 vs. 7.80	0.890	0.827	0.795	0.509	0.741	**0.955**
	7.72 vs. 7.80	0.904	0.764	0.822	0.588	0.743	**0.946**
libz	1.2.3.4 vs.1.2.11	0.632	0.697	0.700	0.520	0.560	**0.800**
	1.2.3.9 vs. 1.2.11	0.637	0.612	0.786	0.599	0.598	**0.833**
	1.2.8 vs. 1.2.11	0.751	0.732	0.839	0.747	0.768	**0.903**
Avg.		0.758	0.708	0.779	0.548	0.658	**0.870**

SUB substitutes specific instructions within a basic block and does not change a function's control-flow graph significantly. With BCF, it adds additional basic blocks into a function in a fixed pattern. With FLA, it flattens the control flow by simply splitting original basic blocks into smaller ones or adding extra basic blocks into an original function.

For each program, we generate three pairs and each pair consists of a CLANG-compiled binary and an OLLVM-compiled binary with one obfuscation option. We then fed each pair into SemDiff and two representative machine-learning-based tools (i.e., Gemini and Palmtree) for similarity quantification. The results are shown in Table 9. Clearly, SemDiff outperforms the other two tools for all obfuscation options by large margin.

To understand the root cause of the failure cases of SemDiff, we manually analyzed the results and found that in experiments of Section 4.3.1 and Section 4.3.2, when SemDiff failed to rank the ground truth similar function at the first place, in approximately 50% of the cases, SemDiff still rank the similar function before 10th place. We consider this still can assist humans to find similar functions efficiently. In the other 50% cases, SemDiff failed to rank similar functions at front positions mainly due to three reasons: 1) Lack of support for some less frequent mnemonics such as `cvtss2sd`. This can negatively impact semantic information extraction thus decreasing precision. 2) Some calls are optimized into other instructions. For example, `call strlen` is replaced with `repne scasb`, which has the same impact and behavior as `call strlen`. Even using symbolic execution, their symbolic values still differ enormously. 3) Sometimes, the unfolded loop and the folded loop can be difficult to match. Because their symbolic expressions can differ. And the loop number does not match (i.e., only one unfolded loop but multiple unfolded loops).

For the experiment in Section 4.3.3, we speculate that although the obfuscation options obfuscate a binary in terms of its syntactic structures, they retain its key semantics, which can be retrieved by SemDiff. For the three evaluated tools, their generated scores under the SUB option achieve the highest compared to the other options. This is probably because the SUB option does not change the control flow. Of the three options, scores in the FLA option are the lowest, as it introduces more syntactical and control-flow changes by flattening the control flow. The failure cases are caused by newly added key instructions by obfuscation, despite the three reasons mentioned in the last paragraph.

4.4. Real-world applications

4.4.1. RQ 5: Cross-program-version similarity detection

In real-world applications, binary code similarity can be utilized to find similar versions of library binaries or executables because vulnerabilities tend to inherit across a range of versions. Therefore, in this section, for two given binaries from the same program with different versions, we quantify their similarities. For each program, four versions spanning from months to years are compiled using GCC 5.4.0. For each program, one version is selected as the baseline version (we note that this version is used in previous experiments of Sections 4.3.1 and 4.3.2.) and its similarity with each of the other three versions is computed using precision@1, generating 39 pairs of binaries in total. Part of the results is displayed in Table 8 due to the page limit. Please refer to our GitHub repository for complete results.

Overall, SemDiff outperforms all the other tools. Particularly, SemDiff achieves the best detection performance in 10 programs and ranks second in the remaining 3 programs, i.e., coreutils, libgmp, and sqlite3. For both coreutils and sqlite3, SemDiff's averaged score is only 0.01 lower than that of Bindiff. For libgmp, SemDiff's score is only 0.03 lower than that of Asm2vec. A possible reason why SemDiff performs less well in the 3 programs is: as the version difference in the 3 programs is smaller than that of the 10 programs, it indicates that the versions in these programs have more similarities in syntactic structures, which are easier to be captured by tools that rely on syntactic and structural

Table 9
The similarity scores of Gemini, Palmtree and SemDiff for programs compiled by CLANG 3.8.0 with *O*0 and OLLVM with different obfuscation options.

Program	OLLVM-SUB			OLLVM-BCF			OLLVM-FLA		
	Gemini	Palmtree	SemDiff	Gemini	Palmtree	SemDiff	Gemini	Palmtree	SemDiff
openssl	0.627	0.713	**0.973**	0.034	0.056	**0.840**	0.004	0.004	**0.444**
libtomcrypt	0.309	0.520	**0.973**	0.021	0.026	**0.849**	0.003	0.004	**0.721**
coreutils	0.410	0.589	**0.835**	0.033	0.037	**0.839**	0.001	0.001	**0.582**
libMagickCore	0.403	0.481	**0.934**	0.011	0.018	**0.885**	0.001	0.001	**0.623**
libMagickWand	0.192	0.129	**0.997**	0.027	0.032	**0.962**	0.001	0.006	**0.733**
libgmp	0.310	0.649	**0.769**	0.073	0.085	**0.715**	0.061	0.066	**0.604**
curl	0.348	0.248	**0.967**	0.055	0.021	**0.922**	0.014	0.021	**0.75**
sqlite3	0.113	0.113	**0.814**	0.011	0.007	**0.941**	0.001	0.001	**0.639**
libz	0.400	0.355	**0.941**	0.073	0.073	**0.963**	0.027	0.027	**0.817**
plink	0.178	0.161	**0.940**	0.019	0.014	**0.644**	0.001	0.002	**0.694**
pscp	0.188	0.170	**0.945**	0.018	0.019	**0.653**	0.001	0.003	**0.689**
psftp	0.186	0.145	**0.936**	0.019	0.014	**0.656**	0.001	0.003	**0.688**
puttygen	0.226	0.195	**0.930**	0.026	0.032	**0.671**	0.006	0.011	**0.718**
Avg.	0.234	0.198	**0.925**	0.032	0.026	**0.779**	0.007	0.010	**0.714**

features. When the version difference becomes larger in other programs, SemDiff performs the best. The reasons for the failure cases in this experiment are also mainly due to lack of support for rare mnemonics, replacing calls to equivalent instructions, and difficulty in precisely matching loops.

4.4.2. RQ 6: Vulnerability detection

An important application of binary code similarity detection is to find similar vulnerable functions. We randomly selected 40 Common Vulnerabilities and Exposures (CVEs) functions and detect their similar vulnerable functions. For each vulnerable function, we randomly select a vulnerable version of it as the base function. We also prepared another randomly selected vulnerable function version, compiled with random compiling settings (i.e., either O0, O1, O2, O3, Os). They are the target functions that the tools should detect as similar to the base function. We mix the target functions with all other functions from the binary and check the probability of the tool successfully ranking the target vulnerable function at the first place among all functions (i.e., top-1 score). The result is shown in Table 12. Gemini's top-1 score is 52.5% (21 out of 40), UniASM's top-1 score is 42.5% (17 out of 40), Asm2Vec's top-1 score is 75.0% (30 out of 40), while SemDiff is 80.0% (32 out of 40).

We manually analyzed the CVEs where SemDiff fails to identify (rank at the first place). We found that out of 8 failure cases, in 6 cases (75%) SemDiff ranked the vulnerable function before 10th place. This still indicates the effectiveness of using SemDiff to find vulnerabilities. For the other 2 failure cases, one is due to IDA pro failing to identify the indirect jump addresses thus making SemDiff unable to produce a complete key-semantics graph. Another case was due to lack of support for less frequent mnemonics, which negatively impact the semantic information extraction and thus decreased the precision.

4.5. Design effectiveness

4.5.1.

RQ 7: Code coverage of key instruction semantic–an empirically effective heuristic

To assess how much of the program semantics is captured by our proposed key instructions, we also conduct a data-flow analysis to track the propagation destination of each non-key instruction. We calculate the percentage of operand values from non-key instructions that ultimately reach a key instruction in each function. We collect the compiled binaries from all the projects in the dataset, each with different optimization levels. For each function used in similarity comparison, we perform a data-flow analysis. Data-flow analysis is performed within each function based on the symbolic execution framework described in Section 3. Consequently, if two distinct instructions reference memory aliases that point to the same physical address, they are resolved to identical symbolic expressions. For example, if the memory address *ebx* in the instruction *mov eax, [ebx]* and *Rtmin* in *mov ecx, [Rtmin]* evaluate to the same value, the symbolic engine maps them to the same symbolic expression. When a register's value is implicitly modified, the engine ensures that the updated symbolic value correctly propagates forward through the control flow. For example, the instruction *mul ebx* explicitly references only *ebx*. However, the symbolic engine strictly follows architecture conventions to ensure the resulting 64-bit value is implicitly split and stored across the *edx:eax* register pair. Specifically, *edx* is mapped to the high 32 bits, while *eax* is assigned the low 32 bits of the multiplication product. The symbolic engine ignores flags because it cannot inherently carry rich data-flow semantics. Rather than tracking these lossy flag states, the engine directly processes the flag-modifying instructions—either key or non-key instructions, such as *test, add, sub, inc, dec, mul, and div*—thereby preserving their full algebraic and relational semantics. Function return values are symbolically represented as *RET_XXX*, where *XXX* denotes the address of the corresponding function call, as outlined in Section 3. This symbolic value is subsequently propagated forward through the remainder of the function. The results are shown in Table 10.

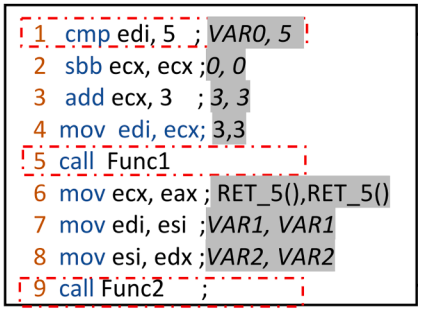


Fig. 5. An example of the code coverage metric.

Our code coverage metric is defined as follows. The numerator is the sum of the key instruction and all non-key instructions whose data flows eventually propagate to key instructions. The denominator is the total number of instructions in the function. The resulting percentage represents the proportion of function semantics captured by our method. Fig. 5 illustrates an example of this coverage metric, where Lines 1, 5, and 9 denote key instructions, and all remaining lines (Lines 2–4 and 6–8) represent non-key instructions. The symbolic expressions evaluated

Table 10
Data-flow analysis of code coverage by SemDiff.

	GCC	CLANG
o0	94.5%	94.8%
o1	95.1%	95.3%
o3	94.7%	95.2%
os	95.0%	95.2%

Table 11
Weighted LSH similarity accuracy for different key instruction types.

	call	compare	indirect	store	accuracy
weights	1	1	1	1	78.15%
weights	2	1	1	1	77.79%
weights	1	2	1	1	78.13%
weights	1	1	2	1	77.82%
weights	1	1	1	2	78.13%

during execution are displayed in gray to the right of each operand. In this scenario, the instructions in Lines 2–4 calculate the single argument *edi* passed to Line 5, while the instructions in Lines 7–8 prepare the two arguments *edi* and *esi* for line 9. Only the instruction on Line 6 fails to propagate to a key instruction within this specific view, resulting in a code coverage score of 8/9 ≈ 88.8%. While a truncated code snippet is shown here for simplicity, the value of *ecx* on Line 6 may propagate further to other key instructions in the latter portion of the complete function.

With different optimization levels, SemDiff retains a similar percentage of program semantics, approximately 95%. To investigate which types of instructions are not propagated to the key instructions, we manually analyzed the results. We found that some non-key instructions, such as prolog instructions (e.g., `push eax; push ebx`), do not propagate their data into the key instructions. Other missed instructions are rarely used, such as `MULX`, `MOVS`, and `LSL`. Since SemDiff is currently a proof of concept, it would be straightforward to incorporate these rarely used instructions in future versions, potentially improving code coverage.

4.5.2.

RQ 8: Weighted LSH accuracy When comparing similarity, SemDiff uses Locality-Sensitive Hashing (LSH). By default, LSH treats each key instruction type with equal weight. However, to explore the impact of assigning different weights to different key instructions, we use weighted LSH for similarity comparison. Specifically, we assign different weights to the symbolic expression tokens of various key instruction types, while leaving the weights of the other key instructions at the default value of 1.0. We randomly select over 10,000 functions from the dataset, which includes functions compiled with different settings, compilers, and versions.

Table 11 shows the overall accuracy of weighted LSH on the selected data. The first row corresponds to the default LSH method, without considering weights. The results indicate that, when different weights are assigned, the accuracy remains approximately the same. This could be due to the use of symbolic execution, which mitigates the impact of low-level address differences.

When more weight is assigned to calling instructions or indirect branches, the accuracy decreases slightly. This may be because these instructions carry less semantic information compared to others (e.g., the majority of calling instructions contain only zero or one parameter.

5. Discussion

Key Expression Accuracy: As analyzed in Section 4, lack of support for less frequent mnemonics can decrease precision. Therefore, more accuracy can increase with more complete mnemonics supports. Due to the time limit and the complexity of all the mnemonics, our current version only supports the most frequently used mnemonics. Future work can add more key instructions for more complete semantics preservation. Apart from complete mnemonics support, adding hard-coded optimization knowledge into SemDiff can also increase the accuracy. As we analyzed before, `call strlen` and `repne scasb` has different symbolic expressions. Identifying them as the same can only be achieved through adding that optimization knowledge into SemDiff. Since GCC and Clang are open-sourced, a promising direction is to parse those source code to learn those knowledge automatically.

Graph Diffing: In our current method, we use LSH algorithm to translate each graph into a hash value. Therefore, we equally consider the importance of each token in the key expression. Also, for each kind of key instruction, we also consider them to have equal importance. However, some tokens and key instructions should have more importance than others. For example, the matching of long instant values should indicate more similarity than matching some frequent operational symbols such as $*$. Matching two calling type key instructions with four arguments should weigh more than one argument. The weight of each token and key instruction type can be learned by machine learning if one prepares adequate training data.

Inherited Limitations: In Section 5, there were cases when the static binary code analysis platform IDA pro that we rely on failed to analyze indirect jump targets. Unfolded and folded loops matching is also not well resolved. One possible solution is to unfold the loop for a fixed time. Another solution is to conclude the unfolded loops into one loop and compare them with the folded loop. As to key expression simplification, SemDiff utilized msynth (Blazytko & Schloegel, 2021), which potentially has time efficiency and accuracy issues.

6. Related work

6.1. Program-analysis based methods

SMIT (Hu et al., 2009), BINCLONE (Farhadi et al., 2014), and SPAIN (Xu et al., 2017b) use hashing techniques to output various instructions sequences into a fixed length of the hash value and compare their similarity. IDEA (Santos et al., 2010), MBC (Kang et al., 2012), Expose (Ng & Prakash, 2013) generate an embedding from sequences. Exediff (Baker et al., 1999), Tracy (David & Yahav, 2014), and Binsequence (Huang et al., 2017) align two sequences and decide their similarity. SMIT (Hu et al., 2009), Binslayer (Bourquin et al., 2013; Cesare et al., 2014) transform the problem into finding the mapping between two CFGs with minimum cost. Beagle (Lindorfer et al., 2012), Cesare et al. (2014), rendezous (Khoo et al., 2013), and FOSSILE (Alrabaee et al., 2018) divide the graph into k subgraphs and match subgraphs similarity. CoP (Luo et al., 2014), SIGMA (Alrabaee et al., 2015), Binsequence (Huang et al., 2017) determine similarity based on paths. Beagle (Lindorfer et al., 2012), FOSSIL (Alrabaee et al., 2018), and SIGMA (Alrabaee et al., 2015) classified instruction based on their arithmetic, logic, or data transfer operations. Binhash (Jin et al., 2012), MULTI-HM (Pewny et al., 2015), Bingo (Chandramohan et al., 2016), SPAIN (Xu et al., 2017b), Kargén and Shahmehri (2017), IMF-SIM (Wang & Wu, 2017) check whether output is the same as the input. Binhunt (Gao et al., 2008), Binhash (Jin et al., 2012), Expose (Ng & Prakash, 2013), CoP (Luo et al., 2014), MULTI-MH (Pewny et al., 2015), ESH (David et al., 2016) symbolically execute the binaries and compare similarity by constraint solver. XMATCH (Feng et al., 2017), TEDEM (Pewny et al., 2014) determine the edit distance of the tree/graph of the symbolic formula. Static function-level methods compare blocks or instructions. Each block or instruction is treated as the smallest similarity unit. However, compiler optimizations can split blocks or replace instructions. As a result, block structures and instructions change com-

Table 12
Vulnerability function search results. ✓represents the tool ranked vulnerable target function at the first place in all functions. ✗is vice versa.

CVE	Gemini	UniASM	Asm2Vec	SemDiff	CVE	Gemini	UniASM	Asm2Vec	SemDiff
Overflow/out of bound					Certification/Authentication				
CVE-2024-9143	✓	✗	✓	✓	CVE-2025-4575	✗	✗	✓	✓
CVE-2024-5535	✗	✗	✓	✓	CVE-2023-28321	✓	✗	✓	✓
CVE-2025-9086	✓	✗	✗	✓	CVE-2020-8286	✗	✗	✓	✗
CVE-2017-1000100	✗	✓	✓	✓	CVE-2023-27538	✗	✗	✓	✓
CVE-2017-1000101	✗	✗	✓	✗	CVE-2023-27536	✓	✗	✗	✓
CVE-2017-1000254	✓	✗	✓	✓	CVE-2023-27535	✗	✓	✓	✓
CVE-2019-5436	✓	✓	✗	✓	CVE-2025-49812	✓	✗	✓	✓
CVE-2017-7407	✓	✗	✓	✓	CVE-2017-2629	✗	✓	✗	✓
CVE-2017-8817	✗	✗	✓	✗	Loop-related				
CVE-2017-1000257	✓	✓	✓	✓	CVE-2020-8285	✓	✓	✓	✓
CVE-2017-9502	✓	✗	✓	✓	CVE-2025-5399	✓	✓	✗	✓
free-related					CVE-2024-4603	✓	✓	✓	✓
CVE-2024-4741	✗	✗	✓	✓	Misc				
CVE-2016-8618	✗	✓	✗	✗	CVE-2024-12797	✓	✗	✓	✓
SSRF					CVE-2024-13176	✗	✗	✓	✓
CVE-2024-43394	✓	✓	✓	✓	CVE-2024-6119	✓	✗	✓	✓
CVE-2024-10524	✗	✗	✗	✓	CVE-2024-2511	✓	✓	✓	✓
Information leakage					CVE-2025-10148	✗	✓	✗	✓
CVE-2023-46218	✗	✗	✓	✓	CVE-2023-28320	✗	✗	✓	✓
CVE-2023-28322	✗	✗	✗	✓	CVE-2016-8617	✓	✓	✓	✗
CVE-2020-8169	✗	✓	✓	✗	CVE-2016-8615	✗	✗	✗	✓
CVE-2021-22876	✓	✓	✓	✗	CVE-2016-8616	✓	✓	✓	✗
					CVE-2023-38546	✓	✓	✓	✓

pletely. Thus, block- or instruction-level feature, or graphical feature comparison is not ideal for binary similarity (Ren et al., 2021).

6.2. Machine-learning based methods

Genius (Feng et al., 2016), Vulseeker (Gao et al., 2018), Gemini (Cochard et al., 2022; Xu et al., 2017a; Yu et al., 2020), TIKNIB (Kim et al., 2022), Asteria-Pro (Yang et al., 2023), Jason et al. (Kim et al., 2023) extract features from graphs into feature vectors and determine the vector similarity. QBinDiff (Mengin & Rossi, 2021)extracts binary code features and uses graph edit distance and network alignment methods to measure similarity. αDiff (Liu et al., 2018), InnerEye (Zuo et al., 2022), Asm2Vec (Ding et al., 2019), Kam1n0 (Ding et al., 2016), J-Trans (Wang et al., 2022), Palmtree (Li et al., 2021), UniASM (Gu et al., 2025), and Safe (Massarelli et al., 2019) automatically learn the embedding for each instruction and use them to produce the basic-level or function-level embedding. As mentioned in Section 1, machine-learning methods work in theory but rely on good training data. Binaries vary widely across compilers and optimization levels [17]. This makes dataset building hard and accuracy unstable. Unsupervised models learn from context but fail under heavy optimization. Supervised models preserve syntax, not full semantics.

7. Conclusion

This paper proposed SemDiff, which is a novel semantic method for binary similarity detection. SemDiff has two modules: graph generation and graph diffing. In graph generation, we proposed complete instructions traversal of a given function and a lightweight loop processing to generate key instruction symbolic expressions. We further translate symbolic expressions to key expressions and form a key-semantics graph. We utilized LSH to diff two key-semantics graphs.

In our evaluation, We compared SemDiff with 5 state-of-the-art baseline tools in binary similarity detection, results of which showed that SemDiff outperformed all the baseline tools on average. While our current version of SemDiff works for x86-based binaries SemDiff can be extended to support similarity detection cross architectures (e.g., x86 and ARM) in our future work. Particularly, we will extend the graph-generation module to extract and transform key instructions from a target architecture into key expressions.

CRediT authorship contribution statement

Zian Liu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation; **Xiaoxi Meng:** Writing – review & editing, Visualization, Validation, Resources, Investigation; **Viet Vo:** Writing – review & editing, Supervision, Project administration, Conceptualization; **Siqi Ma:** Writing – review & editing, Supervision, Methodology; **Chao Chen:** Writing – review & editing, Project administration, Funding acquisition; **Dongxi Liu:** Writing – review & editing, Supervision, Investigation, Formal analysis; **Jun Zhang:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Formal analysis; **Shigang Liu:** Writing – review & editing, Supervision; **Muhammad Ejaz Ahmed:** Writing – review & editing, Supervision; **Yang Xiang:** Writing – review & editing, Project administration, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Alrabaee, S., Shirani, P., Wang, L., & Debbabi, M. (2015). Sigma: A semantic integrated graph matching approach for identifying reused functions in binary code. *Digital Investigation*, *12* (S1), S61–S71.

Alrabaee, S., Shirani, P., Wang, L., & Debbabi, M. (2018). Fossil: A resilient and efficient system for identifying foss functions in malware binaries. *ACM Transactions on Privacy and Security 21* (2), 1–34.

Alshahwan, N., Barr, E. T., Clark, D., Danezis, G., & Menéndez, H. D. (2020). Detecting malware with information complexity. *Entropy*, *22*(5), 575.

Baker, B. S., Manber, U., & Muth, R. (1999). Compressing differences of executable code. In *Acmsigplan workshop on compiler support for system software* (pp. 1–10).

Blazytko, T., & Schloegel, M. (2021). msynth. GitHub GitHub repository https://github.com/mrphrazer/msynth

Borbely, R. S. (2015). On normalized compression distance and large malware towards a useful definition of normalized compression distance for the classification of large files. *Journal of Computer Virology and Hacking Techniques*. *12*, 235–242.

Bourquin, M., King, A., & Robbins, E. (2013). BinSlayer: Accurate comparison of binary executables. In *ACM sigplan program protection and reverse engineering workshop*. Article 4, 1–10.

Cesare, S., Xiang, Y., & Zhou, W. (2014). Control flow-based malware variant detection. *IEEE Transactions on Dependable and Secure Computing*, *11*, 307–317.

Chandramohan, M., Xue, Y., Xu, Z., Liu, Y., Cho, C. Y., & Tan, H. B. K. (2016). BinGo: Cross-architecture cross-OS binary search. In *ACM sigsoft international symposium on foundations of software engineering*.

Cheng, B., Ming, J., Fu, J., Peng, G., Chen, T., Zhang, X., & Marion, J.-Y. (2018). Towards paving the way for large-scale windows malware analysis: Generic binary unpacking with orders-of-magnitude performance boost. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 395–411).

Cochard, V., Pfammatter, D., Duong, C. T., & Humbert, M. (2022). Investigating graph embedding methods for cross-platform binary code similarity detection. In *2022 IEEE 7th European symposium on security and privacy (euros&p)* (pp. 60–73). IEEE.

David, R., Coniglio, L., & Ceccato, M. (2020). Qsynth-a program synthesis based approach for binary code deobfuscation. In *Bar 2020 workshop*.

David, Y., Partush, N., & Yahav, E. (2016). Statistical similarity of binaries. In *PLDI* (p. 266–280).

David, Y., & Yahav, E. (2014). Tracelet-based code search in executables. In *PLDI* (p. 349–360).

Ding, S. H. H., Fung, B., & Charland, P. (2016). Kam1n0: Mapreduce-based assembly clone search for reverse engineering. In *ACM SIGKDD international conference on knowledge discovery and data mining*.

Ding, S. H. H., Fung, B., & Charland, P. (2019). Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization. In *Oakland* (pp. 472–489).

Duan, Y., Li, X., Wang, J., & Yin, H. (2020). DeepBinDiff: Learning program-wide code representations for binary diffing. In *NDSS*

Dullien, T. (2018). Functionsimsearch. https://github.com/googleprojectzero/functionsimsearch. Retrieved October 2026.

Eschweiler, S., Yakdan, K., & Gerhards-Padilla, E. (2016). discovRE: Efficient cross-architecture identification of bugs in binary code. In *NDSS* (pp. 58–79).

Farhadi, M. R., Fung, B. C. M., Charland, P., & Debbabi, M. (2014). Binclone: Detecting code clones in malware. In *SERE* (pp. 78–87).

Feng, Q., Wang, M., Zhang, M., Zhou, R., Henderson, A., & Yin, H. (2017). Extracting conditional formulas for cross-platform bug search. In *AsiaCCS* (p. 346–359).

Feng, Q., Zhou, R., Xu, C., Cheng, Y., Testa, B., & Yin, H. (2016). Scalable graph-based bug search for firmware images. In *CCS* (p. 480–491).

Flake, H. (2004). Structural comparison of executable objects. In *Detection of intrusions and malware & vulnerability assessment, GI SIG SIDAR workshop, DIMVA 2004*.

Gao, D., Reiter, M. K., & Song, D. (2008). BinHunt: Automatically finding semantic differences in binary programs. In *International conference on information and communications security* (pp. 238–255).

Gao, J., Yang, X., Fu, Y., Jiang, Y., & Sun, J. (2018). VulSeeker: A semantic learning based vulnerability seeker for cross-platform binary. In *ASE* (p. 896–899).

Gu, Y., Shu, H., Kang, F., & Hu, F. (2025). UniASM: Binary code similarity detection without fine-tuning. *Neurocomputing*, *630*, 129646.

Haq, I. U., & Caballero, J. (2021). A survey of binary code similarity. *ACM Computing Survey*, *54*(3).

Hu, X., Chiueh, T.-c., & Shin, K. G. (2009). Large-scale malware indexing using function-call graphs. In *CCS* (p. 611–620).

Huang, H., Youssef, A. M., & Debbabi, M. (2017). Binsequence: Fast, accurate and scalable binary code reuse detection. In *AsiaCCS* (p. 155–166).

Jin, W., Chaki, S., Cohen, C., Gurfinkel, A., Havrilla, J., Hines, C., & Narasimhan, P. (2012). Binary function clustering using semantic hashes. In *International conference on machine learning and applications* (p. 386–391).

Junod, P., Rinaldini, J., Wehrli, J., & Michielin, J. (2015). Obfuscator-LLVM – software protection for the masses. In *IEEE/ACM international workshop on software protection* (pp. 3–9).

Kan, Z., Wang, H., Wu, L., Guo, Y., & Luo, D. X. (2019). Automated deobfuscation of android native binary code. arXiv preprint arXiv:1907.06828

Kang, B., Kim, T., Kwon, H., Choi, Y., & Im, E. G. (2012). Malware classification method via binary content comparison. In *RACS* (p. 316–321).

Kargén, U., & Shahmehri, N. (2017). Towards robust instruction-level trace alignment of binary code. In *ASE* (pp. 342–352).

Khoo, W. M., Mycroft, A., & Anderson, R. (2013). Rendezvous: A search engine for binary code. In *Working conference on mining software repositories* (pp. 329–338).

Kim, D., Kim, E., Cha, S. K., Son, S., & Kim, Y. (2022). Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. *IEEE Transactions on Software Engineering*. *49* (4), 1661–1682.

Kim, J., Genkin, D., & Leach, K. (2023). Revisiting lightweight compiler provenance recovery on ARM binaries. In *2023 IEEE/ACM 31st international conference on program comprehension (ICPC)* (pp. 292–303). IEEE.

Li, X., Qu, Y., & Yin, H. (2021). PalmTree: Learning an assembly language model for instruction embedding. In *CCS* (pp. 3236–3251).

Lin, G., Zhang, J., Luo, W., Pan, L., De Vel, O., Montague, P., & Xiang, Y. (2019). Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Transactions on Dependable and Secure Computing*. *18* (5), 2469–2485.

Lindorfer, M., Di Federico, A., Maggi, F., Comparetti, P. M., & Zanero, S. (2012). Lines of malicious code: Insights into the malicious software industry. In *ACM annual computer security applications conference* (p. 349–358).

Liu, B., Huo, W., Zhang, C., Li, W., Li, F., Piao, A., & Zou, W. (2018). αDiff: Cross-version binary code similarity detection with DNN. In *ASE* (p. 667–678).

Luo, L., Ming, J., Wu, D., Liu, P., & Zhu, S. (2014). Semantics-based obfuscation-resilient binary code similarity comparison with applications to software plagiarism detection. In *FSE* (p. 389–400).

Massarelli, L., Luna, G. A. D., Petroni, F., Querzoni, L., & Baldoni, R. (2019). Safe: Self-attentive function embeddings for binary similarity. arXiv: abs/1811.05296

Mengin, E., & Rossi, F. (2021). Binary diffing as a network alignment problem via belief propagation. In *IEEE/ACM international conference on automated software engineering* (pp. 967–978).

Ming, J., Xu, D., Jiang, Y., & Wu, D. (2017). BinSim: Trace-based semantic binary diffing via system call sliced segment equivalence checking. (pp. 253–270).

Nagra, J., & Collberg, C. (2009). Surreptitious software: Obfuscation, watermarking, and tamperproofing for software protection: Obfuscation, watermarking, and tamperproofing for software protection.

Ng, B. H., & Prakash, A. (2013). Expose: Discovering potential binary code re-use. In *IEEE annual computer software and applications conference* (pp. 492–501).

Pewny, J., Garmany, B., Gawlik, R., Rossow, C., & Holz, T. (2015). Cross-architecture bug search in binary executables. In *OAKLAND* (pp. 709–724).

Pewny, J., Schuster, F., Bernhard, L., Holz, T., & Rossow, C. (2014). Leveraging semantic signatures for bug search in binary programs. In *Annual computer security applications conference* (p. 406–415).

Raff, E., & Nicholas, C. (2017a). An alternative to NCD for large sequences, lempel-ziv jaccard distance. In *ACM SIGKDD international conference on knowledge discovery and data mining* (p. 1007–1015).

Raff, E., & Nicholas, C. (2017b). Malware classification and class imbalance via stochastic hashed LZJD. In *ACM workshop on artificial intelligence and security* (pp. 111–120).

Ren, X., Ho, M., Ming, J., Lei, Y., & Li, L. (2021). Unleashing the hidden power of compiler optimization on binary code difference: An empirical study. In *PLDI* (p. 142–157).

Roundy, K. A., & Miller, B. P. (2013). Binary-code obfuscations in prevalent packer tools. *ACM Computing Surveys*, *46*, 1–32.

Saıdi, H., Porras, P., & Yegneswaran, V. (2010). Experiences in malware binary deobfuscation. *Virus Bulletin*.

Santos, I., Brezo, F., Nieves, J., Penya, Y. K., Sanz, B., Laorden, C., & Bringas, P. G. (2010). Idea: Opcode-sequence-based malware detection. In *International symposium on engineering secure software and systems* (Lecture Notes in Computer Science), *5965*, Springer, 35–43..

Wang, H., Qu, W., Katz, G., Zhu, W., Gao, Z., Qiu, H., Zhuge, J., & Zhang, C. (2022). jTrans: Jump-aware transformer for binary code similarity detection. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis* In *ASE* (pp. 1–13).

Wang, S., & Wu, D. (2017). In-memory fuzzing for binary code similarity analysis. In *ASE* (p. 319–330).

Xu, X., Liu, C., Feng, Q., Yin, H., Song, L., & Song, D. (2017a). Neural network-based graph embedding for cross-platform binary code similarity detection. In *CCS* (pp. 363–376).

Xu, Z., Chen, B., Chandramohan, M., Liu, Y., & Song, F. (2017b). Spain: Security patch analysis for binaries towards understanding the pain and pills. In *2017 IEEE/ACM 39th international conference on software engineering (ICSE)* (pp. 462–472). IEEE.

Yamaguchi, F., Lottmann, M., & Rieck, K. (2012). Generalized vulnerability extrapolation using abstract syntax trees. In *Annual computer security applications conference* (p. 359–368).

Yang, S., Dong, C., Xiao, Y., Cheng, Y., Shi, Z., Li, Z., & Sun, L. (2023). Asteria-pro: Enhancing deep learning-based binary code similarity detection by incorporating domain knowledge. *ACM Transactions on Software Engineering and Methodology*, *33*(1), 1–40.

Yu, Z., Cao, R., Tang, Q., Nie, S., Huang, J., & Wu, S. (2020). Order matters: Semantic-aware neural networks for binary code similarity detection. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 1145–1152). (*Vol. 34*).

Zhou, Y., Main, A., Gu, Y. X., & Johnson, H. (2007). Information hiding in software with mixed boolean-arithmetic transforms. In *International workshop on information security applications* (pp. 61–75).

Zuo, F., Li, X., Zhang, Z., Young, P., Luo, L., & Zeng, Q. (2022). Neural machine translation inspired binary code similarity comparison beyond function pairs. In *NDSS*.